

AASHA

SIH 2026 — Technology Stack & AI/ML Architecture

Project: Multi-Layer Health, Nutrition & Locality Analysis Model

Architecture: Mobile-first • Offline-first • Secure • API-based

Purpose: A presentation-ready technical reference for the AASHA SIH 2026 solution.

Executive technical position

AASHA should not be presented as a system that uses one AI model for everything. Its strength is a multi-layer architecture: deterministic rules handle validation, age and anthropometric calculations, nutritional requirements and nutrient-gap calculations; machine learning is reserved for pattern/risk prediction where sufficient quality data exists; Generative AI is used for understandable explanations, multilingual communication and natural-language interaction.

Core implementation stack: Flutter + Dart, SQLite, Python + FastAPI, PostgreSQL, Pydantic, NumPy/Pandas, scikit-learn, a controlled LLM API, REST/JSON, JWT/RBAC and HTTPS/TLS.

1. End-to-End Architecture

Layer	Recommended technology	Primary responsibility
Mobile	Flutter + Dart	Registration, records, dashboards, recommendations, multilingual UI
Offline	SQLite + local sync queue	Store data when connectivity is unavailable
API	Python + FastAPI + REST/JSON	Secure communication between app, database and analysis services
Validation	Pydantic + deterministic rules	Missing, invalid, duplicate, inconsistent and range checks
Analysis	Python + NumPy/Pandas	Health, nutrition, dietary and trend calculations
Database	PostgreSQL	Central health, nutrition, locality, reference and audit data
ML	scikit-learn	Clustering and future prediction/risk models
Recommendation	Python scoring/ranking engine	Rank food/diet recommendations
Generative AI	LLM API	Explain validated results and support language interaction
Security	HTTPS/TLS + JWT + RBAC + audit logs	Protect health-related information

2. AI/ML Model Strategy

AASHA layer	Model / approach	MVP status	Why
Layer 0 — Data Quality	Rule engine + Pydantic	Required	Deterministic validation is safer and auditable
Layer 1 — Demographic	DOB/age rules	Required	Age and life-stage calculation is deterministic
Layer 2 — Anthropometric	Formulae + reference standards	Required	BMI/growth indicators depend on validated standards
Layer 3 — Health/Vitals	Reference rules + trend analysis	Required	Flags and trends should not be framed as diagnosis
Layer 4 — Nutrition	Reference-data/rule engine	Required	Calculate applicable nutrient requirements
Layer 5 — Dietary	Food composition DB + calculations	Required	Estimate nutrient intake from foods consumed
Layer 6 — Nutrient Gap	Requirement – estimated intake	Required	Primarily a calculation layer
Layer 7 — Food	Filtering + weighted ranking	Required	Map nutrient gaps to suitable foods
Layer 8 — Locality	SQL/Python rules + ranking	Required	Local availability, seasonality, affordability
Layer 9 — Community	Pandas + K-Means (optional MVP)	Optional	Identify community dietary/nutritional patterns
Layer 10 — Recommendation	Content-based / weighted recommender	Required	Personalized ranking of recommendations
Layer 11 — Explanation	Controlled LLM API	Required	Convert validated outputs into simple language
Layer 12 — Monitoring	Trend analysis; time-series later	Basic MVP	Track changes in measurements and gaps
Future risk prediction	Logistic Regression / Random Forest / XGBoost	Future	Only after sufficient high-quality labelled data

3. Recommended Tech Stack by Functional Layer

Mobile application

- **Flutter + Dart** for a single mobile codebase.
- Use a maintainable architecture such as UI → ViewModel → Repository → Service/Data layers.
- The same backend API can support a future web client.

Offline-first data

- **SQLite** on the device.
- Maintain a synchronization queue with retry, duplicate prevention and conflict handling.
- Do not depend on live internet for basic registration, record viewing or data entry.

Backend/API

- **Python + FastAPI** with REST/JSON.
- Pydantic models for request/response validation.
- Separate API routes from analysis, recommendation and AI services.

Central database

- **PostgreSQL** for users, persons, measurements, health records, assessments, food data, reference standards, recommendations, synchronization and audit records.
- Keep development/staging/production environments separated.

Data/analytics

- **NumPy** for numerical calculations.
- **Pandas** for tabular analysis and community/trend processing.
- **scikit-learn** for clustering and future ML experiments.

AI/LLM

- Use one controlled **LLM API** behind the FastAPI backend.
- Use it for explanation, multilingual communication and natural-language interaction.
- Do not let the LLM overwrite calculated health/nutrition values or make an uncontrolled medical diagnosis.

Security

- **HTTPS/TLS** for transport.
- **JWT + RBAC** for authentication/authorization.
- Encrypted local storage, audit logs, protected backups and environment separation.

4. Detailed AI/ML Models

A. Rule-Based Health & Nutrition Engine — Core

This is the foundation, not an ML model. It performs data validation, age calculation, anthropometric calculations, reference-range flags, nutrient requirements and nutrient-gap calculations. It should be versioned and traceable to the reference source used.

B. Content-Based Food Recommendation — Core

Start with a deterministic/weighted recommender. Match nutrient gaps to food composition data, then filter by dietary suitability, locality, seasonality and affordability. Rank the remaining foods. This is preferable to training a recommender before AASHA has sufficient user interaction data.

C. K-Means Clustering — Optional community ML

Use only for aggregated, privacy-preserving community analysis. Features may include dietary patterns, nutrient gaps and food-access indicators. The purpose is to identify groups with similar patterns, not to diagnose individuals.

D. Trend / Time-Series Analysis — Core monitoring

Store historical height, weight, blood pressure and other measurements. Start with simple trend calculations and moving averages. More advanced forecasting can be evaluated later if the dataset supports it.

E. Logistic Regression / Random Forest / XGBoost — Future prediction

Potential future models for risk or pattern prediction. These require appropriate labelled, high-quality data, validation, bias testing and clinical/domain review. They should not be claimed as trained medical models in the initial SIH prototype unless the team actually trains and validates them.

F. Generative AI / LLM — Core AI interaction

Use an LLM only after the deterministic analysis pipeline has produced validated results. The LLM converts structured results into understandable explanations, multilingual responses and natural-language interaction. Use strict prompting, structured inputs/outputs and guardrails.

5. Reference Data & Data Strategy

AASHA's model report identifies potential primary sources including ICMR-NIN, Indian Food Composition Tables, WHO, Government of India sources and ABDM. The exact production rule must be tied to the applicable source/version. Reference values should remain traceable to organization, document, version, table/section, value, effective date and import/verification history.

The project currently has no large real-world AASHA dataset. Therefore, the development plan should use authoritative reference data plus clearly labelled synthetic data. Real production data must remain separated from development/staging data.

6. AI Safety & Explainability Rules for SIH

- **AI does not diagnose.** Automated health outputs must not be represented as medical diagnoses.
- **LLM does not calculate authoritative values.** Requirements and classifications come from validated rules/reference data.
- **Trace every analysis.** Store analysis ID, person/assessment ID, layer, engine/model version, input version, reference source/version, result, confidence where applicable and timestamp.
- **Protect individual privacy.** Community analysis should use aggregated information and avoid exposing individual health records.
- **Separate environments.** Development uses synthetic data; production contains real data only.
- **Human/clinical oversight.** High-impact health decisions should remain subject to appropriate professional review.

7. API / Service Structure

Service	Example endpoints	Purpose
Auth	/api/auth/*	Login, token refresh, roles
Persons	/api/persons/*	Registration and profile
Measurements	/api/measurements/*	Historical height/weight/BP etc.
Health	/api/health/*	Health indicators and flags

Service	Example endpoints	Purpose
Nutrition	/api/nutrition/*	Requirements and nutrient gaps
Foods	/api/foods/*	Food composition and availability
Analysis	/api/analysis/*	Run multi-layer analysis
Recommendations	/api/recommendations/*	Return ranked recommendations
Community	/api/community/*	Aggregated community analytics
Sync	/api/sync/*	Offline-to-server synchronization
AI	/api/ai/*	Controlled LLM explanation requests

8. What the SIH MVP Should Actually Implement

- **Must have:** Flutter mobile UI, SQLite offline storage, sync queue, FastAPI backend, PostgreSQL, authentication/RBAC, data validation, anthropometric calculations, nutrition requirement engine, nutrient-gap engine, food database, locality-aware ranking and LLM explanation.
- **Good differentiator:** K-Means clustering over aggregated synthetic community data, with a clear explanation of what each cluster represents.
- **Do later:** trained medical risk prediction, sophisticated collaborative filtering, advanced forecasting and large-scale personalization. These depend on sufficient quality data.
- **Demo flow:** Register person → enter measurements/diet → validate → calculate nutrition → identify nutrient gap → filter local foods → rank recommendations → generate simple multilingual explanation → save offline → synchronize when online.

9. SIH Presentation — One-Line Technical Pitch

“AASHA is an offline-first, secure, multi-layer health and nutrition platform that combines deterministic evidence-based analysis, locality-aware recommendation algorithms, privacy-preserving community ML and controlled Generative AI explanations through a common API.”

10. Technology Selection Rationale

Flutter is suitable for the mobile-first requirement and supports multi-platform development from a single codebase. Its official architecture guidance separates UI, data/repository and optional domain layers, which fits AASHA's modular design.

FastAPI is a Python API framework with automatic OpenAPI documentation and Pydantic-based validation, making it a practical backend choice for a system combining APIs with Python data/ML components.

These are implementation recommendations for AASHA, not claims that SIH mandates these specific technologies.

Sources & project basis

Primary project basis: AASHA Model Report V1.0 (15 August 2026), supplied with this task.

Official Flutter documentation: Flutter supports multi-platform applications from a single codebase and provides guidance for app architecture and AI integration.

Official FastAPI documentation: FastAPI is a Python framework for APIs with automatic interactive documentation/OpenAPI support.